

Содержание

Предисловие	2
Модель вычислений RAM	3
Анализ сложности наилучшего, наихудшего и среднего случая	4
Big O нотация	5
Определения	5
Операции с O	7
Скорость роста	8
Графики основных асимптотических оценок	9
Вывод точной оценки на примере	10
Примеры для оценки асимптотики алгоритмов по времени	11
Константное время: $O(1)$	11
Пример 1.	11
Пример 2.	11
Пример 3.	11
Логарифмическое время: $O(\log n)$	12
Пример 4.	12
Пример 5.	13
Пример 6.	13
Квадратный корень: $O(\sqrt{n})$	14
Пример 7.	14
Пример 8.	14
Линейное время: $O(n)$	15
Пример 9.	15
Пример 10.	15
Пример 11.	15
Линейно-логарифмическое время: $O(n \log n)$	16
Пример 12.	16
Пример 13.	16
Полуторная степень: $O(n\sqrt{n})$	17
Пример 14.	17
Пример 15.	17
Квадратичное время: $O(n^2)$	18
Пример 16.	18
Пример 17.	18
Пример 18.	19
Кубическое время: $O(n^3)$	20
Пример 19.	20
Пример 20.	20
Функции от нескольких переменных	22
Пример 21.	22
Пример 22.	22
Скользящее окно	23
Пример 23.	23
Простая рекурсия	24
Пример 24. Линейная рекурсия: $O(n)$	24
Пример 25. Логарифмическая рекурсия: $O(\log n)$	24
Пример 26. Квадратичная рекурсия с циклом внутри: $O(n^2)$	24
Практика	25

Предисловие

Небольшая методичка по асимптотике, которая поможет лучше разобраться в теме.

Материалы взяты из:

- The Art of Computer Programming, Donald E. Knuth
- Big Omicron and Big Omega and Big Theta, Donald E. Knuth
- The Algorithm Design Manual, Steven S. Skiena

Алгоритмы являются принципиально важным компонентом информатики, поскольку их изучение не требует использования языка программирования или компьютера. Это означает необходимость в методах, позволяющих сравнивать эффективность алгоритмов, не прибегая к их реализации. Самыми значимыми из этих инструментов являются модель вычислений RAM и асимптотический анализ вычислительной сложности.



Модель вычислений RAM

Разработка машинно-независимых алгоритмов основывается на гипотетическом компьютере, называемом *машиной с произвольным доступом к памяти* (Random Access Machine), или RAM-машиной. Согласно этой модели наш компьютер работает таким образом:

- для исполнения *любой* простой операции: (+, *, -, =, if, call) требуется ровно 1 временной шаг;
- циклы и подпрограммы не считаются простыми операциями, а состоят из нескольких простых операций;
- каждое обращение к памяти занимает один временной шаг. Кроме этого, наш компьютер обладает неограниченным объемом оперативной памяти. Кэш и диск в модели RAM не применяются.

Время исполнения алгоритма в RAM-модели вычисляется по общему количеству шагов, требуемых алгоритму для решения задачи.

Может показаться, что RAM-модель является слишком упрощенным представлением компьютеров:

- на большинстве процессоров умножение занимает больше времени, чем сложение;
- благодаря оптимизации компилятором и гиперпоточности процессора можно значительно ускорить выполнение циклов и подпрограмм
- время обращения к данным может разниться в зависимости от расположения данных в иерархии памяти.

Таким образом, по сравнению с настоящим компьютером, все три основные допущения для RAM-машины неверны.

Тем не менее, несмотря на эти несоответствия настоящему компьютеру, RAM-модель является превосходной моделью для понимания того, как алгоритм будет работать на настоящем компьютере. Она обеспечивает хороший компромисс, отражая поведение компьютеров и одновременно являясь простой в использовании.

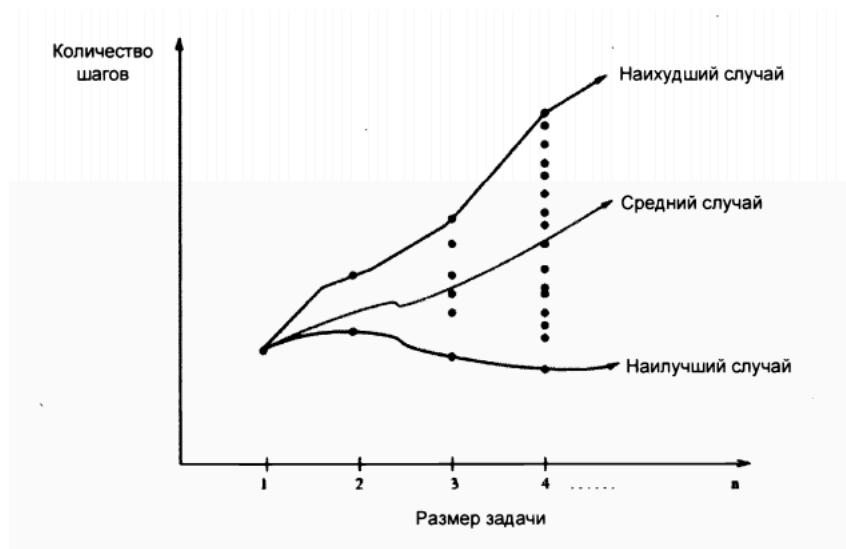
Довольно трудно создать алгоритм, для которого RAM-модель выдаст существенно неверные результаты. Устойчивость этой модели позволяет анализировать алгоритмы машинно-независимым способом.

Анализ сложности наилучшего, наихудшего и среднего случая

С помощью RAM-модели можно подсчитать количество шагов, требуемых алгоритму для исполнения любого экземпляра задачи. Но чтобы получить общее представление о том, насколько хорошим или плохим является алгоритм, нам нужно знать, как он работает со *всеми* возможными экземплярами задачи.

Чтобы понять, что означает наилучший, наихудший и средний случаи, нужно рассмотреть исполнение алгоритма на всех возможных экземплярах входных данных.

Рассмотрим задачу сортировки массива и построим график XOY , где ось OX представляет размер массива, а ось OY — количество шагов, необходимых алгоритму для обработки соответствующего массива:



Точки (x_i, y_i) выстраиваются столбцами, так как размер массива должен быть натуральным числом, как и количество шагов.

Если в каждом из таких столбцов соединить все верхние (по оси OY) точки, то мы получим линию, которая показывает максимально возможное число шагов для каждого размера данных. Эта линия описывает наихудший случай выполнения алгоритма.

Если соединить самые нижние точки в каждом столбце, то мы получим линию, которая показывает минимально необходимое число шагов для каждого размера данных. Эта линия описывает наилучший случай. Она показывает идеальные условия, при которых алгоритму «везет» больше всего, и определяет границу, ниже которой реальное число шагов никогда не опустится.

Если же с помощью инструментов математической статистики определить среднее значение точек для каждого столбца и соединить их, то мы получим линию среднего случая. Эта линия отражает поведение алгоритма на практике, когда входные данные распределены случайно. Она наиболее полезна для оценки реального времени работы программы, хотя рассчитать её математически сложнее всего.

Необходимо строго разграничивать данные понятия: «случай» (наихудший, наилучший или средний) определяет специфику входных данных, в то время как соответствующие им кривые на графике представляют собой функции временной или пространственной сложности. Эти функции количественно описывают затраты ресурсов алгоритма в рамках заданного сценария (случая).

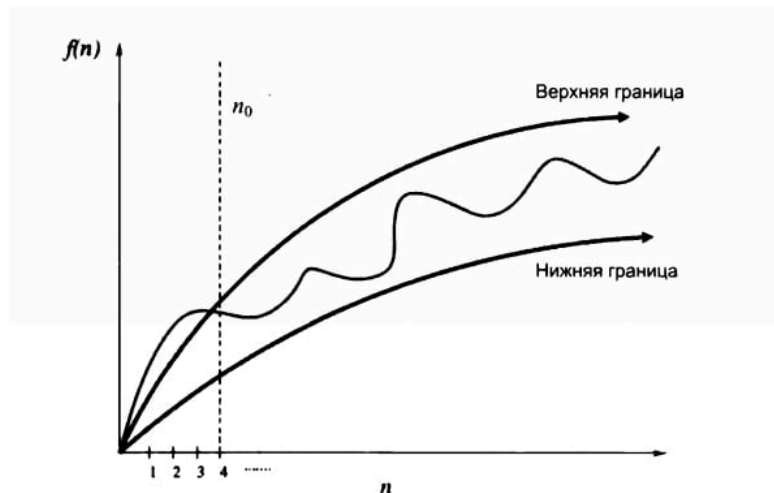
Big O нотация

Определения

Временную (а также пространственную) сложность наихудшего, наилучшего и среднего случаев для любого алгоритма можно представить как числовую функцию от размера входных данных. Но работать с такими функциями очень трудно, поскольку:

- их графики часто имеют локальные экстремумы;
- они требуют слишком много информации для точного вычисления.

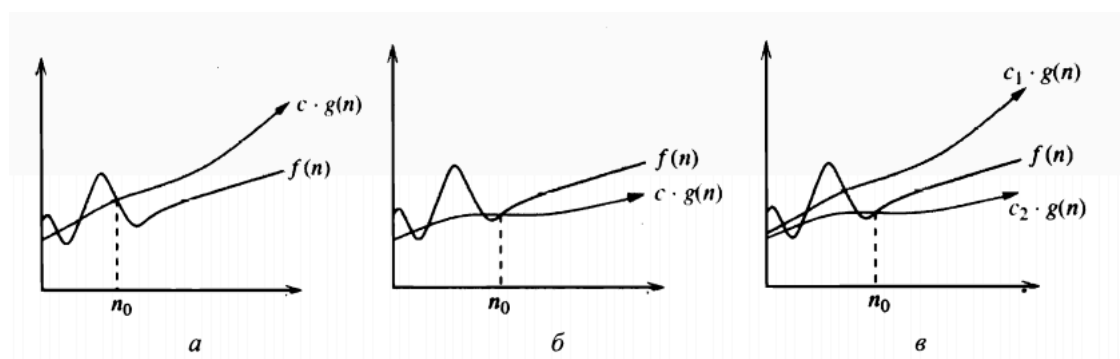
На практике работать с такой точной функцией неудобно. Гораздо проще «зажать» её между двумя гладкими границами — верхней и нижней.



Верхняя и нижняя границы, действительные для $n > n_0$, сглаживают волнистость сложных функций

Для математического описания этих границ вводят специальные асимптотические оценки.

Рассмотрим функцию $f(n)$ от количества входных данных n , поданных на вход алгоритма, значения которой – количество абстрактных единиц времени (или памяти), затрачиваемых алгоритмом:



а. Оценка сверху: $f(n) = O(g(n))$

$$\exists c, n_0 > 0 : \forall n \geq n_0 \rightarrow 0 \leq f(n) \leq c \cdot g(n)$$

б. Оценка снизу: $f(n) = \Omega(g(n))$

$$\exists c, n_0 > 0 : \forall n \geq n_0 \rightarrow 0 \leq c \cdot g(n) \leq f(n)$$

с. Точная оценка: $f(n) = \Theta(g(n)) \Leftrightarrow f(n) \in (O(g(n)) \cap \Omega(g(n)))$

$$\exists c_1, c_2, n_0 > 0 : \forall n \geq n_0 \rightarrow 0 \leq c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$$

Пояснения к оценкам выше:

- а. Оценка сверху, $f(n) = O(g(n))$, означает, что функция $f(n)$ *растет не быстрее*, чем $g(n)$, умноженную на некоторую константу $c > 0$, начиная с некоторого номера n_0 .
- б. Оценка снизу, $f(n) = \Omega(g(n))$, означает, что функция $f(n)$ *растет не медленнее*, чем $g(n)$, умноженную на некоторую константу $c > 0$, начиная с некоторого номера n_0 .
- с. Точная оценка, $f(n) = \Theta(g(n))$ означает, что функция $f(n)$ *одного порядка роста* с функцией $g(n)$, с точностью до некоторых констант $c_1, c_2 > 0$.

Важно понимать, что

$O(g(n))$, $\Omega(g(n))$ и $\Theta(g(n))$ – это множества функций, а не сами функции.

Например, $O(g(n))$ – это множество всех функций, которые растут не быстрее $g(n)$ с точностью до константы. Поэтому математически корректная запись – $f(n) \in O(g(n))$ (функция f принадлежит множеству $O(g(n))$), а не $f(n) = O(g(n))$.

Однако в литературе и на практике исторически закрепилась запись со знаком равенства: $f(n) = O(g(n))$. Это злоупотребление обозначениями, которое означает «функция $f(n)$ принадлежит классу $O(g(n))$ ». Такая запись удобнее в длинных выкладках, например:

$$T(n) = 2T(n/2) + O(n)$$

что означает $\exists f \in O(n) : T(n) = 2T(n/2) + f(n)$.

Будем придерживаться традиции писать через знак равенства, но важно помнить, что за ним скрывается *отношение принадлежности* функции к множеству функций.

Рассмотрим пример для наглядности: $\frac{1}{2}n^2 + n = O(n^2)$, что означает

$$\exists f \in O(n^2) : f(n) = \frac{1}{2}n^2 + n,$$

что также можно уточнить, используя определение оценки сверху:

$$\exists c, n_0 > 0 : \forall n \geq n_0 \rightarrow 0 \leq \frac{1}{2}n^2 + n \leq \frac{1}{2}n^2 + n^2 = \frac{3}{2}n^2$$

$$n_0 = 1, c = \frac{3}{2}$$

На этом примере можно заметить удобство оценок, в частности, оценки сверху: она позволяет понять главный порядок роста функции на бесконечности, то есть, начиная с размера входных данных $n_0 = 1$, наша функция $f(n) = \frac{1}{2}n^2 + n$ будет ограничена сверху $c \cdot g(n) = \frac{3}{2}n^2$.

Стоит еще раз обратить внимание на то, что $O(g(n))$, в отличие от $\Theta(g(n))$, не дает тот же порядок роста, а лишь указывает границу сверху. Однако не всегда для функции сложности удаётся получить точную оценку $\Theta(g(n))$. Причин две: либо она неизвестна (доказана только верхняя граница), либо её не существует — например, если функция не имеет единого порядка роста. В таких случаях ограничиваются оценкой сверху.

Важное замечание насчет точной оценки

В анализе алгоритмов часто встречается ошибочное утверждение:

$$f(n) = \Theta(g(n)) \Leftrightarrow \Omega(g(n)) = O(g(n))$$

Говорить о равенстве $O(g(n)) = \Omega(g(n))$ абсолютно некорректно, поскольку это принципиально разные множества функций.

Эта путаница обычно возникает из-за неверной интерпретации корректной записи:

$$f(n) = \Omega(g(n)) \text{ and } f(n) = O(g(n))$$

Данное выражение с логическим оператором **and** абсолютно верно. Оно означает, что функция $f(n)$ одновременно принадлежит обоим множествам, то есть зажата между ними:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

Однако из того, что одна конкретная функция $f(n)$ лежит в пересечении этих множеств ($f(n) \in O(g(n)) \cap \Omega(g(n))$), ни в коем случае нельзя делать вывод, что сами множества равны между собой.

Операции с O

Вот некоторые простые операции, которые можно выполнить с O :

- $f(n) = O(f(n))$
- $c \cdot O(f(n)) = O(f(n))$
- $O(O(f(n))) = O(f(n))$
- $O(f(n))O(g(n)) = O(f(n)g(n))$
- $O(f(n)g(n)) = f(n)O(g(n))$

Все эти правила – несимметричные равенства, а правила упрощения, читаемые только слева направо. Попробуем прочитать их в обратную сторону. Возьмём два верных утверждения:

$$n^2 \in O(n^2), \quad n \in O(n^2).$$

Если бы знак $=$ в записи O вёл себя как обычное равенство, мы могли бы «приравнять» правые части:

$$O(n^2) = n^2, \quad O(n^2) = n \implies n^2 = n,$$

что, разумеется, неверно. Ошибка здесь в том, что $O(n^2)$ – это множество функций, а не одна функция. Из того, что два элемента принадлежат одному множеству, не следует, что они равны.

Скорость роста

Используя асимптотические обозначения, мы пренебрегаем постоянными множителями, не учитывая их при вычислении функций. При таком подходе функции

$f(n) = 0,001n^2$ и $g(n) = 1000n^2$ для нас одинаковы, несмотря на то, что значение функции $g(n)$ в миллион раз больше значения функции $f(n)$ для любого n .

Причина, по которой нам достаточно грубого анализа, приводится в таблице ниже, в которой приведены наиболее распространенные функции и их значения для нескольких значений n .

Здесь можно увидеть время исполнения $f(n)$ операций алгоритмов на быстродействующем компьютере, выполняющем операции за одну наносекунду (10^{-9} секунд).

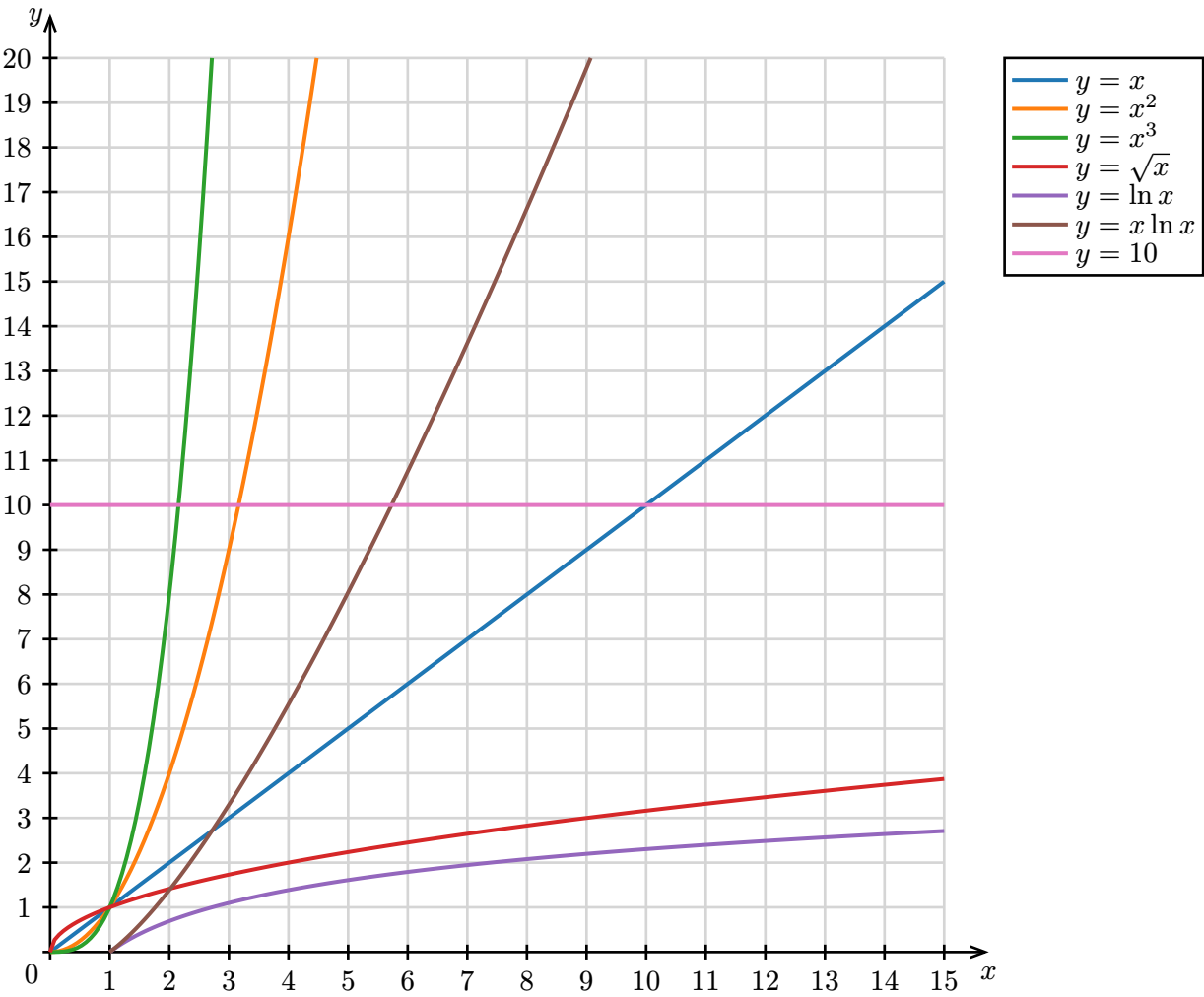
$n/f(n)$	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10	0,003 мкс	0,01 мкс	0,033 мкс	0,1 мкс	1 мкс	3,63 мс
20	0,004 мкс	0,02 мкс	0,086 мкс	0,4 мкс	1 мс	77,1 лет
30	0,005 мкс	0,03 мкс	0,147 мкс	0,9 мкс	1 с	$8,4 \cdot 10^{15}$ лет
40	0,005 мкс	0,04 мкс	0,213 мкс	1,6 мкс	18,3 мин	
50	0,006 мкс	0,05 мкс	0,282 мкс	2,5 мкс	13 дней	
100	0,007 мкс	0,1 мкс	0,644 мкс	10 мкс	$4 \cdot 10^{13}$ лет	
1000	0,010 мкс	1,00 мкс	9,966 мкс	1 мс		
10 000	0,013 мкс	10 мкс	130 мкс	100 мс		
100 000	0,017 мкс	0,10 мс	1,67 мс	10 с		
1 000 000	0,020 мкс	1 мс	19,93 мс	16,7 мин		
10 000 000	0,023 мкс	0,01 с	0,23 с	1,16 дней		
100 000 000	0,027 мкс	0,10 с	2,66 с	115,7 дней		
1 000 000 000	0,030 мкс	1 с	29,90 с	31,7 лет		

Из этой таблицы можем сделать несколько выводов:

- время исполнения всех этих алгоритмов примерно одинаково для значений $n = 10$;
- любой алгоритм с временем исполнения $n!$ становится бесполезным для значений $n \geq 20$;
- диапазон алгоритмов с временем исполнения 2^n несколько шире, но они также становятся непрактичными для значений $n > 40$;
- алгоритмы с квадратичным временем исполнения n^2 применяются при $n \leq 10000$, после чего их производительность начинает резко ухудшаться. Эти алгоритмы, скорее всего, будут бесполезны для значений $n > 1000000$;
- алгоритмы с линейным и логарифмическим временем исполнения остаются полезными при обработке миллиарда элементов;
- в частности, алгоритм $O(\lg n)$ без труда обрабатывает любое вообразимое количество элементов.

Из этого можно сделать основной вывод, что, даже игнорируя постоянные множители, мы получаем превосходное общее представление о годности алгоритма для решения задачи определенного размера.

Графики основных асимптотических оценок



Вывод точной оценки на примере

```
void func(int n) {  
    for (int i = 0; i <= n; ++i) {  
        int y = i;  
        while (y > 1) {  
            y /= 2;  
        }  
    }  
}
```

Внутренний цикл зависит от внешнего, распишем сумму итераций внутреннего:

$$\log(1) + \log(2) + \log(3) + \dots + \log(n-1) + \log(n) = \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-1) \cdot n) = \log(n!)$$

Заметим, что с $O(\log(n!))$ не удобно работать: как сравнить алгоритмы, работающие за $O(\log(n!))$ и $O(n^2)$?

Попробуем дать точную оценку алгоритму: $\Theta(g(n))$.

$$f(n) = \log 1 + \log 2 + \dots + \log n$$

Оценим $f(n)$ сверху:

$$f(n) = \log 1 + \log 2 + \dots + \log n \leq \log n + \log n + \dots + \log n = n \log n$$

$$f(n) \leq n \log n$$

$$f(n) = O(n \log n)$$

Оценим $f(n)$ снизу:

$$f(n) = \log 1 + \log 2 + \dots + \log \frac{n}{2} + \dots + \log n \geq \log \frac{n}{2} + \dots + \log n$$

$$f(n) \geq \left(\frac{n}{2} + 1\right) \log \frac{n}{2} \geq \frac{n}{2} \log \frac{n}{2}$$

$$f(n) \geq \frac{n}{2} (\log n - \log 2) = \frac{n}{2} \log n - \frac{n}{2} \log 2$$

Оценим второе слагаемое сверху. При $n \geq 4$ выполнено $\log 2 \leq \frac{1}{2} \log n$, поэтому:

$$\frac{n}{2} \log 2 \leq \frac{n}{4} \log n$$

Подставим это в предыдущее неравенство:

$$f(n) \geq \frac{n}{2} \log n - \frac{n}{4} \log n = \frac{n}{4} \log n$$

Итак, при $n \geq 4$:

$$f(n) \geq \frac{1}{4} n \log n$$

Следовательно, $f(n) = \Omega(n \log n)$.

Теперь, когда мы доказали, $f(n)$ ограничена и сверху, и снизу функциями одного порядка роста, можно записать точную оценку: $f(n) = \Theta(n \log n)$.

Или же (более формально): $f(n) \in \Theta(n \log n)$.

Примеры для оценки асимптотики алгоритмов по времени

Константное время: $O(1)$

Пример 1.

```
int func(int n) {  
    int res = n * 2 + 5;  
    return res;  
}
```

Пояснение: Время работы не зависит от величины числа n .

Пример 2.

```
int func(int arr[], int index) {  
    return arr[index];  
}
```

Пояснение: Операция доступа к памяти по адресу (указатель + смещение) занимает $O(1)$.

Пример 3.

```
int func(int n) {  
    return n * (n + 1) / 2;  
}
```

Пояснение: Вместо подсчета через цикл используется формула суммы арифметической прогрессии.

Логарифмическое время: $O(\log n)$

Пример 4.

```
int func(int n) {  
    int count = 0;  
    while (n > 0) {  
        count++;  
        n /= 2;  
    }  
    return count;  
}
```

Пояснение: На каждой итерации цикла переменная n уменьшается в два раза. Выделим последовательность:

$$x_n = \left\lfloor \frac{n_0}{2^n} \right\rfloor$$

Как пример, $n_0 = 100$ (n на входе алгоритма).

$$x_1 = \left\lfloor \frac{100}{2} \right\rfloor = 50, \quad x_2 = \left\lfloor \frac{100}{2^2} \right\rfloor = 25, \quad x_3 = \left\lfloor \frac{100}{2^3} \right\rfloor = 12, \quad x_4 = \left\lfloor \frac{100}{2^4} \right\rfloor = 6,$$

$$x_5 = \left\lfloor \frac{100}{2^5} \right\rfloor = 3, \quad x_6 = \left\lfloor \frac{100}{2^6} \right\rfloor = 1, \quad x_7 = \left\lfloor \frac{100}{2^7} \right\rfloor = 0$$

Алгоритм закончил свое выполнение за 7 итераций.

Теперь рассмотрим в общем виде и выведем асимптотику:

$$x_k = \left\lfloor \frac{n_0}{2^k} \right\rfloor$$

Алгоритм работает, пока $x_k > 0$. Округление вниз становится равным 0 в тот момент, когда знаменатель (2^k) становится больше числителя (n_0):

$$2^k > n_0$$

Теперь выразим k , чтобы явно вычислить количество итераций работы алгоритма:

$$\ln(2^k) > \ln(n_0)$$

$$k \ln 2 > \ln n_0$$

$$k > \frac{\ln n_0}{\ln 2}$$

$$k > \log_2 n_0$$

$$k = \lfloor \log_2 n_0 \rfloor + 1$$

Заметим, что $\log_2 n_0 = \frac{\ln n_0}{\ln 2} = \frac{1}{\ln 2} \cdot \ln n_0$, а $\frac{1}{\ln 2}$ является константой.

Это объясняет, почему в контексте асимптотики часто не пишут основание у логарифма:

$$f(n) = O(\log n).$$

Пример 5.

```
void func(int n) {  
    for (int i = 1; i < n; i *= 2) {  
        // Тело цикла  
    }  
}
```

Пояснение: Переменная цикла i растет экспоненциально (1, 2, 4, 8, 16...).
Алгоритм прекращает работу при $2^k \geq n_0$, где k – количество итераций:

$$\ln(2^k) \geq \ln(n_0)$$

$$k \ln 2 \geq \ln n_0$$

$$k \geq \frac{\ln n_0}{\ln 2}$$

$$k \geq \log_2 n_0$$

$$k = \lceil \log_2 n_0 \rceil$$

$$f(n) = O(\log n)$$

Пример 6.

```
void func(int n) {  
    while (n > 1) {  
        n /= 3;  
    }  
}
```

Пояснение: Значение n уменьшается в три раза на каждом шаге.

Асимптотика равна $O(\log_3 n)$. Поскольку $\log_3 n = \frac{\log_2 n}{\log_2 3}$, а константы в O -нотации отбрасываются, это эквивалентно $O(\log n)$.

Квадратный корень: $O(\sqrt{n})$

Пример 7.

```
int func(int n) {  
    int count = 0;  
    for (int i = 1; i * i <= n; ++i) {  
        count += i;  
    }  
    return count;  
}
```

Пояснение: Условие остановки цикла $i * i \leq n$, что эквивалентно $i \leq \sqrt{n}$. Цикл сделает ровно $\lfloor \sqrt{n} \rfloor$ итераций.

Пример 8.

```
void func(int n) {  
    int i = 2;  
    while (i * i < n) {  
        i += 2;  
    }  
}
```

Пояснение: Несмотря на то, что шаг равен +2, верхняя граница по-прежнему определяется условием $i^2 < n$, то есть $i < \sqrt{n}$. Константа шага уменьшает число итераций вдвое, но асимптотика остается $O(\frac{1}{2}\sqrt{n}) = O(\sqrt{n})$.

Линейное время: $O(n)$

Пример 9.

```
int func(int* arr, int size) {  
    int sum = 0;  
    for (int i = 0; i < size; ++i) {  
        sum += arr[i];  
    }  
    return sum;  
}
```

Пояснение: Цикл проходит n итераций (размер массива).

$$f(n) = O(n).$$

Пример 10.

```
void func(int n) {  
    for (int i = 0; i < n; i += 5) {  
        // Операции  
    }  
}
```

Пояснение: Количество итераций цикла равно $\lceil \frac{n}{5} \rceil$.

$$f(n) = O\left(\frac{1}{5} \cdot n\right) = O(n)$$

Пример 11.

```
void func(int n) {  
    for (int i = 0; i < n; ++i) {}  
    for (int j = 0; j < n; ++j) {}  
}
```

Пояснение: Алгоритм состоит из двух идущих друг за другом линейных циклов. Суммарное время работы равно

$$O(n) + O(n) = O(2n) = O(n)$$

Линейно-логарифмическое время: $O(n \log n)$

Пример 12.

```
void func(int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 1; j < n; j *= 2) {  
            // Тело цикла  
        }  
    }  
}
```

Пояснение: Внешний цикл выполняется ровно n раз. При каждой его итерации запускается внутренний цикл, который делает $\log_2 n$ шагов. Общее число операций вычисляется как произведение:

$$O(n) \cdot O(\log n) = O(n \log n)$$

Пример 13.

```
void func(int n) {  
    for (int i = 1; i < n; i *= 2) {  
        for (int j = 0; j < n; ++j) {  
            // Тело цикла  
        }  
    }  
}
```

Пояснение: Внешний цикл делает $\log_2 n$ шагов, а внутренний на каждой его итерации выполняет ровно n итераций. Итоговая сложность:

$$O(\log n) \cdot O(n) = O(n \log n)$$

Полуторная степень: $O(n\sqrt{n})$

Пример 14.

```
void func(int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 1; j * j <= n; ++j) {  
            // Тело цикла  
        }  
    }  
}
```

Пояснение: Внешний цикл работает за линию ($O(n)$). Внутренний цикл на каждом шаге внешнего работает до корня из n , выполняя $\sqrt{n}$ итераций. В итоге:

$$O(n) \cdot O(\sqrt{n}) = O(n^{\frac{3}{2}})$$

Пример 15.

```
void func(int n) {  
    for (int i = 1; i * i <= n; ++i) {  
        for (int j = 0; j < n; ++j) {  
            // Операции  
        }  
    }  
}
```

Пояснение: Внешний цикл выполняет $\sqrt{n}$ итераций, на каждой из которых внутренних выполняет n итераций. Итоговая асимптотика:

$$O(\sqrt{n}) \cdot O(n) = O(n^{\frac{3}{2}})$$

Квадратичное время: $O(n^2)$

Пример 16.

```
void func(int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 0; j < n; ++j) {  
            // Тело цикла  
        }  
    }  
}
```

Пояснение: На каждой итерации внешнего цикла, работающего за линейку, внутренний выполняет n итераций. В итоге:

$$O(n) \cdot O(n) = O(n^2)$$

Пример 17.

```
void func(int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = i; j < n; ++j) {  
            // Тело цикла  
        }  
    }  
}
```

Пояснение: Внутренний цикл зависит от индекса внешнего. Распишем итерации:

$$i = 0 : j = 0, 1, \dots, n - 1 \quad (n \text{ итераций})$$

$$i = 1 : j = 1, 2, \dots, n - 1 \quad (n - 1 \text{ итераций})$$

$$i = 2 : j = 2, 3, \dots, n - 1 \quad (n - 2 \text{ итераций})$$

...

$$i = n - 1 : j = n - 1 \quad (1 \text{ итерация})$$

Заметим, что сумма итераций образует арифметическую прогрессию:

$$S_n = \frac{1 + n}{2}n = \frac{n^2}{2} + \frac{1}{2}n$$

$$f(n) = O\left(\frac{n^2}{2} + \frac{1}{2}n\right) = O(n^2)$$

Пример 18.

```
void func(int* matrix, int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 0; j < n; ++j) {  
            // Чтение матрицы [i][j]  
        }  
    }  
}
```

Пояснение: Для обработки двумерного массива (матрицы) размером $n \times n$ алгоритму необходимо посетить каждую ячейку. Общее количество элементов равно n^2 .

Кубическое время: $O(n^3)$

Пример 19.

```
void func(int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 0; j < n; ++j) {  
            for (int k = 0; k < n; ++k) {  
                // Тело цикла  
            }  
        }  
    }  
}
```

Пояснение: Каждый из трех вложенных циклов выполняется ровно n раз. Общее количество операций составляет $O(n) \cdot O(n) \cdot O(n) = O(n^3)$.

Пример 20.

```
void func(int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 0; j < i; ++j) {  
            for (int k = 0; k < j; ++k) {  
                // Тело цикла  
            }  
        }  
    }  
}
```

Пояснение:

Внешний цикл по переменной i выполняется n раз.

Средний цикл по переменной j выполняется i раз.

Внутренний цикл по переменной k выполняется j раз.

$$\underbrace{1 + 1 + 1 + \dots + 1}_{j \text{ итераций внутреннего цикла}} = 1 \cdot j = j$$

Теперь суммируем j по всем значениям среднего цикла, от $j = 0$ до $j = i - 1$:

$$\sum_{j=0}^{i-1} j = 0 + 1 + 2 + \dots + (i-1) = \frac{0 + i - 1}{2} \cdot i = \frac{1}{2}i^2 - \frac{1}{2}i$$

Нашли суммарное количество итераций, происходящее за 1 итерацию внешнего цикла.

Теперь найдем итоговое количество итераций:

$$\sum_{i=0}^{n-1} \left(\frac{1}{2}i^2 - \frac{1}{2}i \right) = \frac{1}{2} \left(\sum_{i=0}^{n-1} i^2 - \sum_{i=0}^{n-1} i \right)$$

Рассмотрим суммы отдельно:

Воспользуемся утверждением, что

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} \quad \text{— можно доказать через мат. индукцию.}$$

Тогда $\sum_{i=0}^{n-1} i^2 = 1^2 + 2^2 + \dots + (n-1)^2 = \frac{(n-1)((n-1)+1)(2(n-1)+1)}{6} = \frac{(n-1)n(2n-1)}{6}$

Вторая сумма — сумма арифметической прогрессии:

$$\sum_{i=0}^{n-1} i = 0 + 1 + 2 + \dots + (n-1) = \frac{0 + n-1}{2} \cdot n = \frac{1}{2}n^2 - \frac{1}{2}n$$

Итого:

$$\begin{aligned} \frac{1}{2} \left(\sum_{i=0}^{n-1} i^2 - \sum_{i=0}^{n-1} i \right) &= \frac{1}{2} \left(\frac{(n-1)n(2n-1)}{6} - \left(\frac{1}{2}n^2 - \frac{1}{2}n \right) \right) = \frac{1}{2} \left(\frac{2n^3 - 3n^2 + n}{6} - \frac{1}{2}n^2 + \frac{1}{2}n \right) = \\ &= \frac{1}{2} \left(\frac{1}{3}n^3 - \frac{1}{2}n^2 + \frac{1}{6}n - \frac{1}{2}n^2 + \frac{1}{2}n \right) = \frac{1}{6}n^3 - \frac{1}{2}n^2 + \frac{1}{3}n \end{aligned}$$

В асимптотической нотации:

$$O\left(\frac{1}{6}n^3 - \frac{1}{2}n^2 + \frac{1}{3}n\right) = O(n^3).$$

Функции от нескольких переменных

Пример 21.

```
void func(int a, int b) {  
    for (int i = 0; i < a; ++i) {}  
    for (int j = 0; j < b; ++j) {}  
}
```

Пояснение: Сложность зависит от двух независимых входных величин a и b . Циклы выполняются последовательно, асимптотики складываются:

$$O(a) + O(b) = O(a + b)$$

Пример 22.

```
void func(int a, int b) {  
    for (int i = 0; i < a; ++i) {  
        for (int j = 0; j < b; ++j) {  
            // Операции  
        }  
    }  
}
```

Пояснение: Сложность зависит от двух независимых входных величин a и b . Один цикл вложен в другой, асимптотики переумножаются:

$$O(a) \cdot O(b) = O(a \cdot b)$$

Скользящее окно

Пример 23.

```
void func(int n) {  
    int right = 0;  
    for (int left = 0; left < n; ++left) {  
        while (right < n && right < left + 5) {  
            right++;  
        }  
    }  
}
```

Пояснение:

Можно ошибочно предположить, что раз один цикл вложен в другой, то асимптотика – $O(n^2)$. Однако, переменная `right` инкрементируется не более n раз за все время работы функции. Поэтому итоговая асимптотика – $O(n)$.

Простая рекурсия

Пример 24. Линейная рекурсия: $O(n)$

```
void func(int n) {  
    if (n <= 0) return;  
    func(n - 1);  
}
```

Пояснение: Функция вызывает саму себя, уменьшая аргумент на 1 на каждом шаге до тех пор, пока он не станет равен 0. Формируется цепочка из n последовательных вызовов. Глубина стека и временная сложность растут линейно.

Пример 25. Логарифмическая рекурсия: $O(\log n)$

```
void func(int n) {  
    if (n <= 1) return;  
    func(n / 2);  
}
```

Пояснение: На каждом рекурсивном шаге размер задачи уменьшается в два раза. Это рекурсивный аналог цикла деления `while(n > 0) n /= 2`. Общее количество вызовов до базового случая равно $\log_2 n$.

Пример 26. Квадратичная рекурсия с циклом внутри: $O(n^2)$

```
void func(int n) {  
    if (n <= 0) return;  
    for (int i = 0; i < n; ++i) {  
        // Линейный цикл  
    }  
    func(n - 1);  
}
```

Пояснение: Функция делает n рекурсивных шагов в глубину, но на каждом уровне выполняется обычный цикл `for` длиной в текущее значение n . Суммарное количество операций вычисляется как сумма прогрессии: $n + (n - 1) + (n - 2) + \dots + 1 = O(n^2)$.

Практика

1. $O(n) + O(n) = O(2n) = O(n)$

```
for (int i = 0; i < n; ++i) {  
    ...  
}  
  
for (int j = 0; j < n; ++j) {  
    ...  
}
```

2. $O(\log(\frac{n}{2})) = O(\log n)$

```
for (int i = 0; i < n / 2; i *= 2) {  
    ...  
}
```

3. $O(n) + O(\log n) = O(n)$

```
int i = 1;  
while (i < n) {  
    i *= 5;  
    ...  
}  
for (int i = 0; i < n; ++i) {  
    ...  
}
```

4. $O(n^3)$

```
int j = 0;  
for (int i = 0; i < n / 2; ++i) {  
    while (j < n) {  
        j += 1;  
        for (int k = 0; k < n / 2; ++k) {  
            ...  
        }  
    }  
}
```

5. $O((\log n)^2)$

```
int i = 1;  
while (i < n) {  
    i *= 3;  
    for (int j = 0; j < n; j *= 2) {  
        ...  
    }  
}
```

6. $O(n^2 \log(\sqrt{n})) \rightarrow O(n^2 \log n)$

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        for (int k = 1; k < sqrt(n); k *= 2) {
            ...
        }
    }
}
```

7. $O\left(\left(\frac{n}{4}\right)^2\right) \rightarrow O(n^2)$

```
for (int i = 0; i < n / 4; ++i) {
    for (int j = 0; j < n / 4; ++j) {
        ...
    }
}
```

8. $O(\sqrt{n})$

```
for (int i = 0; i < sqrt(n); ++i) {
    ...
}
```

9. $O(n^2)$

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < i; ++j) {
        ...
    }
}
```

10. $O(\log n \cdot \log n)$

```
int i = 2;
while (i < n) {
    i *= 3;
    int j = 2;
    while (j < i) {
        j *= 2;
        ...
    }
}
```

11. $O(n + \log n + n^2) = O(n^2)$

```
for (int i = 0; i < n; ++i) {  
    ...  
}  
int k = 1;  
while (k < n) {  
    k *= 3;  
    ...  
}  
for (int i = 0; i < n; ++i) {  
    for (int j = 0; j < n / 3; ++j) {  
        ...  
    }  
}
```

12. $O(2^n)$ экспоненциальная сложность: рекурсия или цикл с удвоением.

```
int count = 0;  
for (int i = 0; i < n; ++i) {  
    for (int j = 0; j < (1 << i); ++j) { // 2^i  
        ...  
    }  
}
```

